

Часть А. Set_9

Комкова Полина БПИ244

Номера посылок:

1. A1m - 375444871
2. A1q - 375621042
3. A1r - 375630086
4. A1rq - 375464585

Ссылка на репозиторий: https://github.com/KomokPol/algorithms_set-9

Этап 1. Подготовка тестовых данных

- Разработан генератор строк в зависимости от типа:
 - **0** - рандомно сгенерированный (`random`)
 - **1** - в обратном порядке (`reverse`)
 - **2** - почти отсортированный, где рандомно переставлены 10 пар строк (`almost`)

Для удобства подготовки тестовых данных для каждого случая генерируется массив максимальной длины (3000), из которого выбирается подмассив необходимого размера (100, 200, 300, ... строк)

Этап 2. Эмпирический анализ стандартных алгоритмов сортировки

- Мною были зафиксированы:
 - **Количество посимвольных сравнений**
 - **Время выполнения алгоритмов**

```
for (int r = 0; r < runs; ++r) {
    std::vector<std::string> data = input;
    cmp_count = 0;

    auto t0 = std::chrono::high_resolution_clock::now();
    RunSort(data, sort_id);
```

```

auto t1 = std::chrono::high_resolution_clock::now();

double dt = std::chrono::duration<double, std::micro>(t1 - t0).count();
times.push_back(dt);
cmps.push_back(static_cast<double>(cmp_count));
}

```

- Время вычислялось в микросекундах. После происходило усреднение значений (медиана) при замере в 9 запусков каждого алгоритма

Стандартные StdQuickSort и StdMergeSort написаны вручную для подсчета количество посимвольных сравнений. Каждое сравнение строк инкрементирует глобальный счетчик `cmp_count`. Подсчет вынесен в функцию `CompareStrings` из `common.cpp`:

```

int CompareStrings(const std::string& a, const std::string& b) {
    size_t i = 0;
    while (i < a.size() && i < b.size()) {
        ++cmp_count;
        if (a[i] < b[i]) return -1;
        if (a[i] > b[i]) return 1;
        ++i;
    }
    ...
}

```

Этап 3. Эмпирический анализ адаптированных алгоритмов сортировки

Алгоритмы:

Тернарный STRING QUICK SORT (A1q)

На вход функция получает массив и индекс символа `l`. Сначала отделяются строки, длина которых равна `l`, тк они больше не участвуют. Массив делится на три части по сравнению `l`-го символа с опорным (`pivot` = середина массива):

```

for (std::string str : arr_active) {
    ++cmp_count;
    if (str[l] < pivot[l]) {

```

```

        arr_less.push_back(str);
    } else if (str[l] > pivot[l]) {
        arr_great.push_back(str);
    } else {
        arr_equal.push_back(str);
    }
}

arr_less = TernaryQuickSort(arr_less, l);
arr_equal = TernaryQuickSort(arr_equal, l + 1);
arr_great = TernaryQuickSort(arr_great, l);

```

Идея: средняя часть рекурсивно сортируется уже по следующему символу $l+1$, благодаря чему общий префикс не пересматривается повторно

STRING MERGE SORT с LCP (A1m)

Алгоритм работает с парами (x, k) , где x - строка, k - длина общего префикса с соседом в текущем слиянии. При слиянии для двух текущих $p[i]$ и $q[j]$ сравнение упрощается:

```

if (p[i].second > q[j].second) {
    res.push_back({p[i].first, p[i].second}); ++i;
} else if (p[i].second < q[j].second) {
    res.push_back({q[j].first, q[j].second}); ++j;
} else {
    std::pair<int, int> x_and_h = LcpCompare(p[i].first, q[j].first,
    p[i].second);
    ...
}

```

Важно: полное посимвольное сравнение через `LcpCompare` вызывается только когда k - значения совпадают. В остальных случаях решение принимается без сравнений символов вообще

MSD RADIX SORT (A1r)

Поразрядная сортировка от старшего символа. На каждом уровне:

1. Закончившиеся строки длины l откладываются в `r_excl`

2. Остальные раскладываются по CountingSort по символу `arr[l]`
3. Внутри каждого из 128 бакетов запускается рекурсия на `l + 1`

```
std::vector<std::string> MSBRadixSort(const std::vector<std::string>& str_arr,
int l) {
    if (str_arr.size() <= 1) {
        return str_arr;
    }
    ...
}
```

Важно: сравнений символов здесь нет, поэтому `cmp_count` для этого алгоритма всегда 0. База рекурсии `size <= 1` важна, тк без нее для каждой строки в каждом бакете создавался бы новый `vector<vector<string>>(128)` на каждом уровне вглубь по длине строки, что сильно замедляет работу

MSD RADIX + STRING QUICK SORT (A1rq)

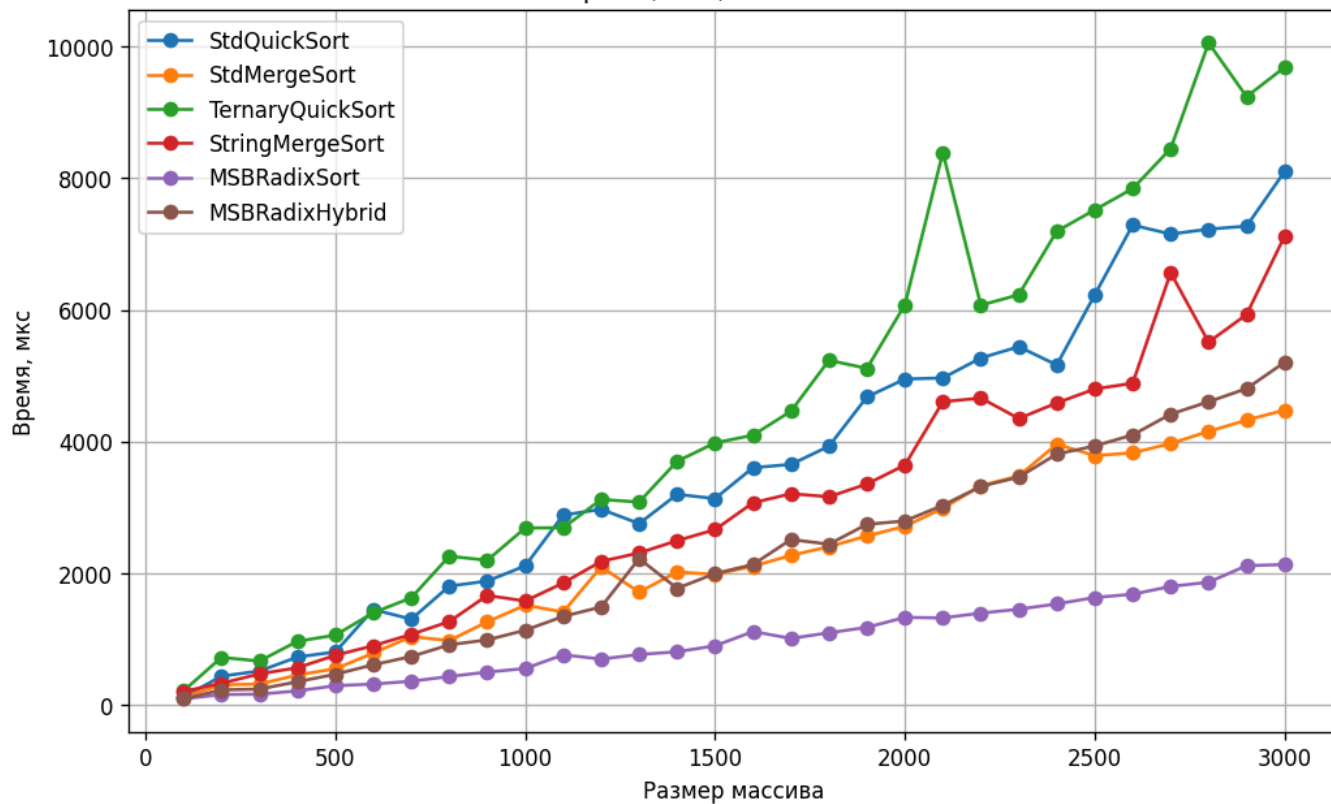
То же самое, но в начале добавлена проверка:

```
if (str_arr.size() < 74) {
    return TernaryQuickSort(str_arr, l);
}
```

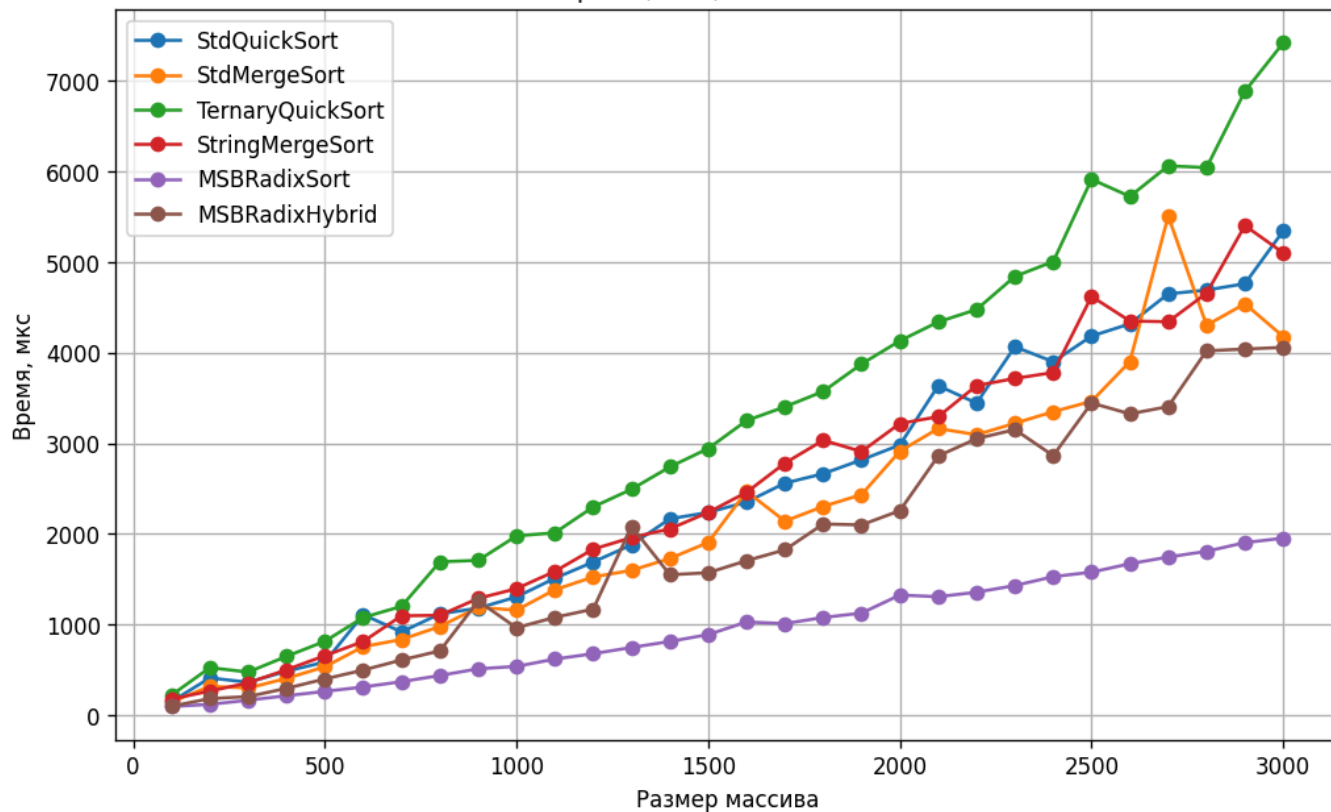
Сравнительный анализ

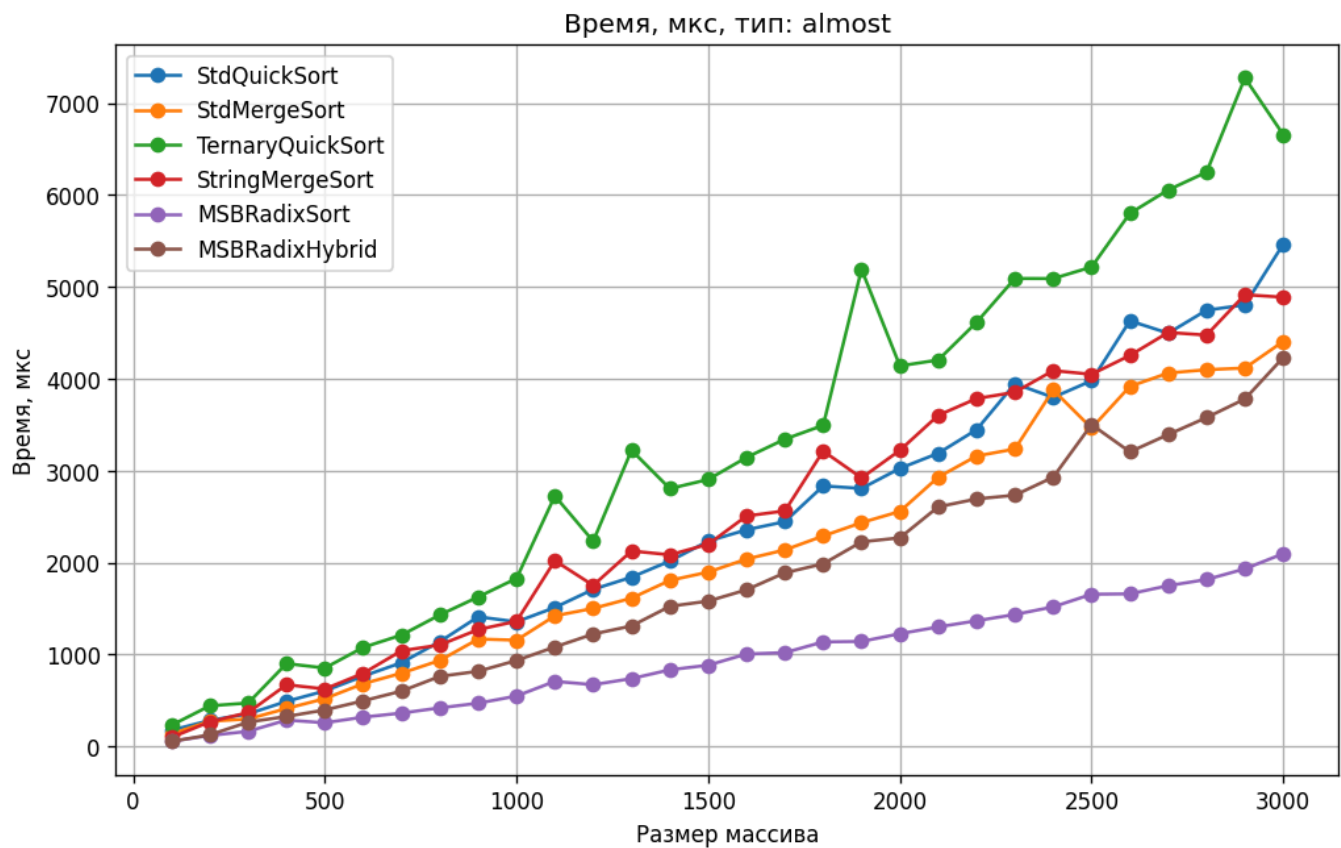
Графики: время выполнения

Время, мкс, тип: random

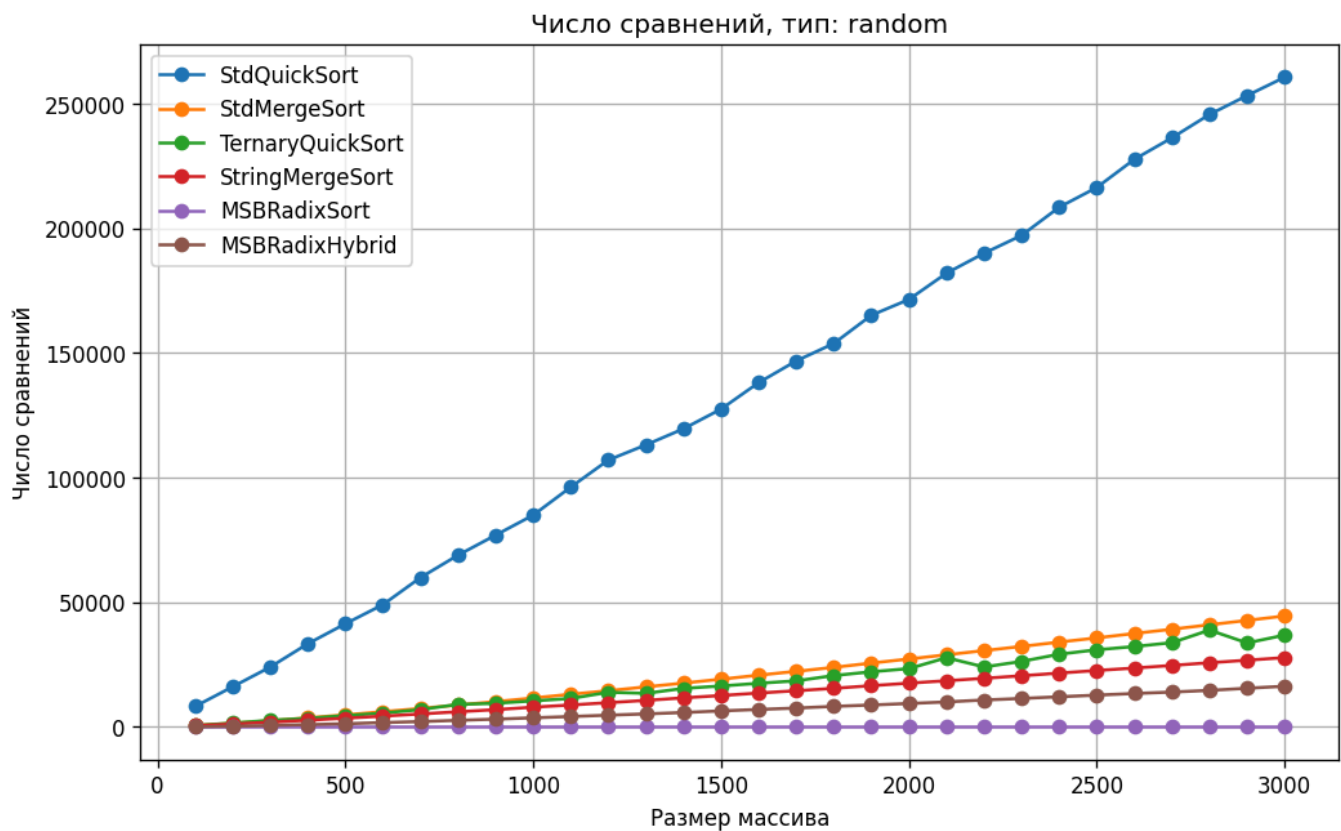


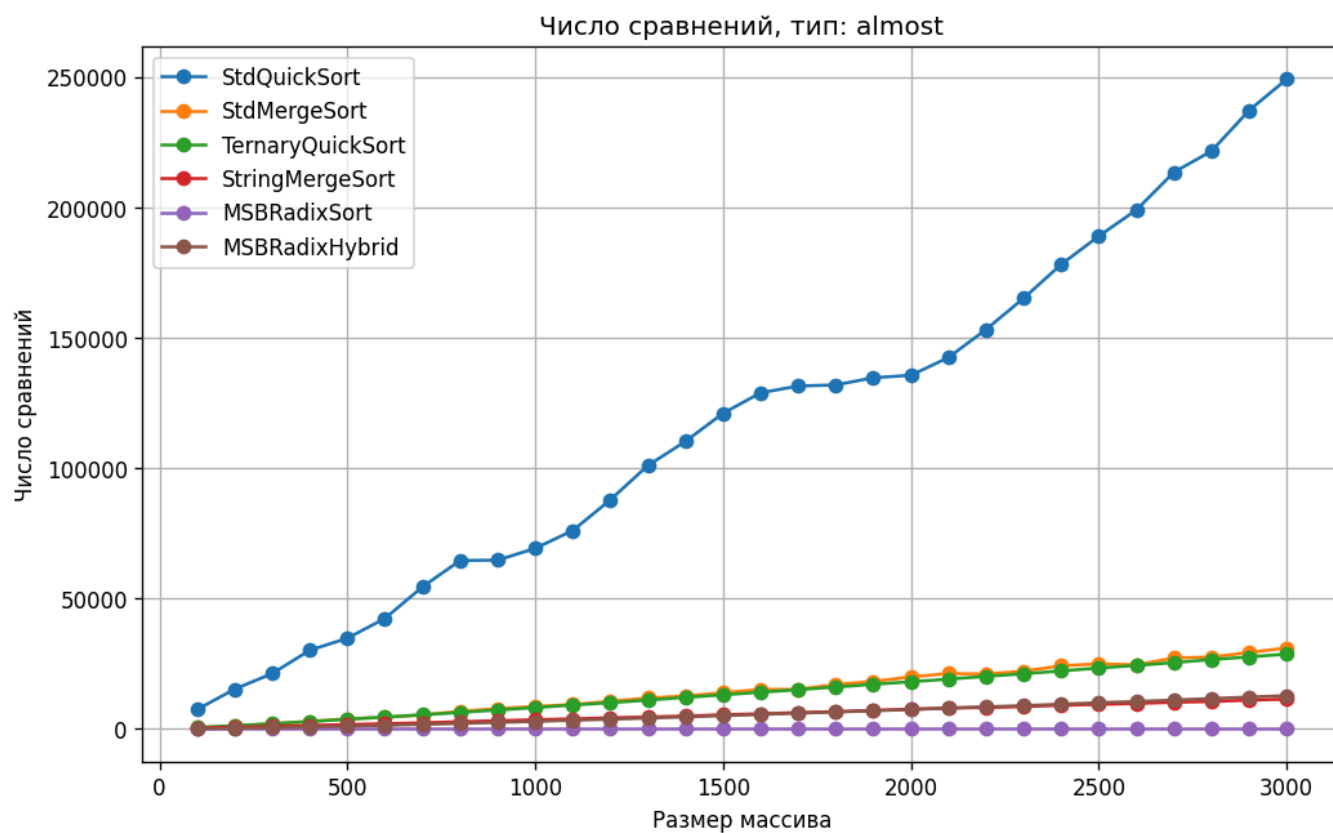
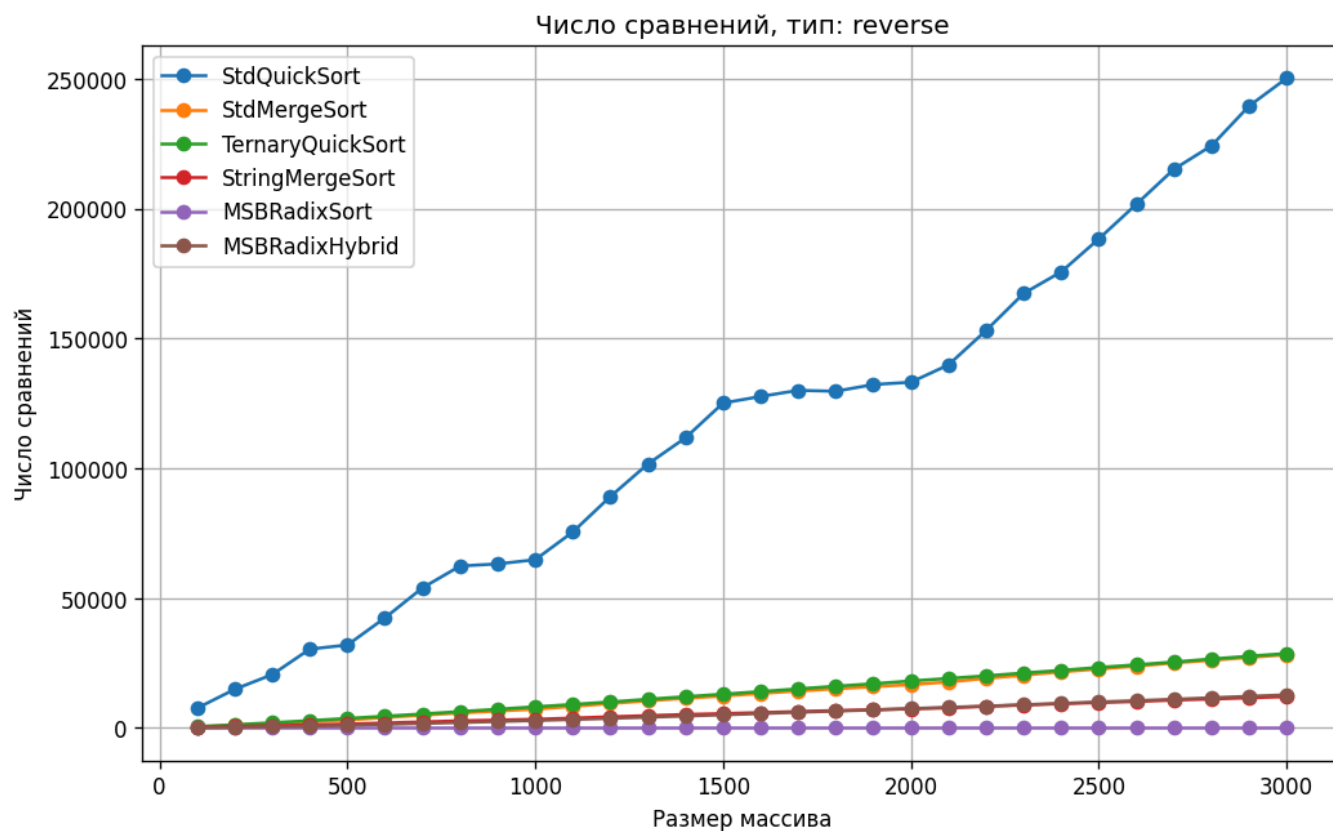
Время, мкс, тип: reverse





Графики: число посимвольных сравнений





Что видно по числу сравнений

- `StdQuickSort` имеет больше всего сравнений, тк алгоритм делает $\Theta(n \cdot \log n)$ полных сравнений строк, каждое из которых проходит до первого расхождения
- `TernaryQuickSort` и `StringMergeSort` дают меньше сравнений. Тк первый не пересравнивает общий префикс между сравнениями, а второй вообще пропускает сравнения, когда LCP-значения соседних кандидатов уже различаются
- `MSBRadixSort` дает 0 сравнений, тк алгоритм работает не на них, а на прямом вычислении позиции по символу
- `MSBRadixHybrid` имеет небольшое число сравнений, тк они происходят только в моменты переключения на тернарный quicksort для подмассивов меньше 74

Что видно по времени

- `MSBRadixSort` самый быстрый алгоритм, тк его сложность $O(N)$, где N - суммарная длина строк, и при достаточно большом алфавите рекурсия быстро разделяет массив на маленькие бакеты, в каждом из которых строки сразу разделяются
- `MSBRadixHybrid` уступает чистому радиксу. Переключение на тернарный quicksort при $|R| < 74$ срабатывает на глубоких уровнях рекурсии, и тернарка на этих подмассивах оказывается медленнее, чем продолжение обычного радикса. Для нашего алфавита массив разбивается на единичные строки достаточно быстро и радикс успевает закончить работу раньше, чем переключение могло бы дать выигрыш
- `StdMergeSort` обгоняет адаптированный `StringMergeSort` и `TernaryQS` по времени. Это так, потому что у случайных строк из алфавита 74 символов **средний LCP очень короткий**, поэтому каждое сравнение в `CompareStrings` отрабатывает очень быстро
- `StdQuickSort` и `TernaryQuickSort` самые медленные. У стандартного QS причина в большом числе глубоких сравнений, у тернарного в том, что копирование подмассивов и рекурсия по три ветви создают много расходов на коротких префиксах

Сопоставление с теоретическими оценками

Пусть n = размер массива, d = средняя длина строки, $|\Sigma|$ = мощность алфавита (74), $N = n \cdot d$ = суммарная длина

- `StdQuickSort`: $O(n \cdot \log n \cdot d)$ сравнений символов. На графиках это самая высокая кривая сравнений, растёт линейно с n при фиксированном $\log n$
- `StdMergeSort`: $O(n \cdot \log n \cdot d)$, но даёт заметно меньше сравнений, тк merge делает меньше повторных сравнений одних и тех же пар

- TernaryQuickSort : $O(n \cdot \log n + N)$. Число сравнений растёт линейно с n и на порядок меньше обычного QS
- StringMergeSort : $O(n \cdot \log n + N)$. Здесь наименьшее число сравнений среди алгоритмов на сравнениях
- MSBRadixSort : $O(N)$. По сравнениям даёт 0, по времени лучший
- MSBRadixHybrid : $O(N)$, но с константой, зависящей от точки переключения. В нашем случае немного медленнее чистого радикаса

Влияние типа массива

- На случайном массиве время работы у всех алгоритмов выше, чем на reverse и almost , тк у близко расположенных строк в реверсном/почти отсортированном порядке после первой же раскладки получаются короткие подмассивы с одинаковыми первыми символами

Выводы

1. По сравнениям MSBRadix - StringMergeSort = TernaryQS - StdMergeSort - StdQuickSort . Адаптированные алгоритмы лучше по сравнениям относительно стандартного quicksort
2. По времени лидер MSBRadixSort и это соответствует теоретической сложности $O(N)$ и подтверждает, что для строк с разнообразными префиксами лучше поразрядная сортировка

Замечания

1. Правила измерения:
 - **Обеспечьте минимальное влияние фоновых процессов и сетевого подключения на результаты измерений**
Я закрыла браузер, мессенджеры, отключила wifi и vpn
 - **Выполните многократные замеры времени работы алгоритмов и рассчитайте среднее значение (конкретное количество замеров и метод усреднения выбираете самостоятельно)**
Я делала 9 раз замеры на подмассиве и усредняла результаты с помощью МЕДИАНЫ, потому что она не зависит от экстремальных значений, которые,

например, учитывает среднее значение. То есть она лучше отражает типичное время выполнения и хорошо работает при наличии скачков

- Правильно подберите единицы измерения времени — слишком крупные единицы (например, секунды) могут дать трудно интерпретируемые результаты

Я выбрала *микросекунды*, потому что это мне показалось самым оптимальным вариантом

2. Отдельный класс `StringSortTester` для реализации функций измерения производительности

```
StringSortTester::Result StringSortTester::Measure(std::vector<std::string>
input, int sort_id, int runs) {
    std::vector<double> times;
    std::vector<double> cmps;

    for (int r = 0; r < runs; ++r) {
        std::vector<std::string> data = input;
        cmp_count = 0;

        auto t0 = std::chrono::high_resolution_clock::now();
        RunSort(data, sort_id);
        auto t1 = std::chrono::high_resolution_clock::now();

        double dt = std::chrono::duration<double, std::micro>(t1 - t0).count();
        times.push_back(dt);
        cmps.push_back(static_cast<double>(cmp_count));
    }

    std::sort(times.begin(), times.end());
    std::sort(cmps.begin(), cmps.end());

    Result res;
    res.avg_time_us = times[runs / 2];
    res.avg_cmp = cmps[runs / 2];
    return res;
}

void StringSortTester::RunSort(std::vector<std::string>& data, int sort_id) {
    if (sort_id == 0) {
        data = StdQuickSort(data);
    } else if (sort_id == 1) {
        data = StdMergeSort(data);
    } else if (sort_id == 2) {
        data = TernaryQuickSort(data, 0);
    }
}
```

```
} else if (sort_id == 3) {  
    auto pairs = StringMergeSort(data);  
    data.clear();  
    for (auto& p : pairs) {  
        data.push_back(p.first);  
    }  
} else if (sort_id == 4) {  
    data = MSBRadixSort(data, 0);  
} else if (sort_id == 5) {  
    data = MSBRadixSortHybrid(data, 0);  
}  
}
```